

保护零：从答案生成到事实成立

生成式 AI 工程中的错误吸收态、独立证据与事实治理

——《从工具增益到协同复利》的理论续篇

作者：然（NoctilumeDev）

文稿性质：论文体工程复盘·理论续篇·归档修订版

归档修订：2026 年 8 月

“模型可以产生答案，但不能拥有事实。”

体例说明

本文承接《从工具增益到协同复利》的乘法模型，但不重复比较具体模型，也不把下列公式包装成严格定理。文中的概率、状态转移和独立性指标主要承担机制解释功能；它们尚未经过大样本估计。案例只对作者在八个仓库归档验收、对抗性复现和单机资源边界中的真实经历负责。

摘要

前文《从工具增益到协同复利》将有效工程产出解释为人、模型、工作流、协议、有效上下文、历史资产与任务类型共同作用的结果，并把判断力与停止能力视为强模型时代的关键乘数。本文继续追问一个更基础的问题：当人的任务定义、模型的概率生成和验收者的复核都可能带有噪声时，一个工程声明凭什么取得“事实”资格？本文以八个公开仓库的对抗性收尾、零上下文审查、真实浏览器与数据库读回、发布物复现、GitHub 治理检查以及单机资源停止线为案例材料，提出四个相互关联的概念。第一，生成者、测试作者和审查者若共享同一错误前提，错误状态会成为近似吸收态；系统长期可靠性不仅取决于降低从正确态进入错误态的概率，更取决于提高从错误态返回正确态的“错误状态逃逸概率”。第二，“人类判断力兜底”并不天然构成可靠性方案，人类复核只有在介入频率、错误发现能力、正确修复能力与证据独立性同时成立时，才会提高纠错概率。第三，大模型不是纯粹的认知放大器，而是带生成项的放大器；当使用者认知为零时，模型先验会填充空白，并可能把未经验证的内容包装成高置信度的“伪正数”。第四，可信系统必须保护 UNKNOWN，把 VERIFIED、UNVERIFIED 与 REFUTED 作为不同状态，并由数据库、浏览器、运行时、Git 对象、发布物和陌生审查者等独立事实源完成状态转换。本文进一步提出“概率性认知核心 + 确定性执行系统 + 持久状态 + 外部事实校验”的两层结构：底层执行操作系统约束 AI 能做什么、记录它实际做了什么；上层验收工作台判断行动后的世界究竟变成了什么。本文不宣称消灭幻觉，也不认为确定性等于正确性；其目标只是让幻觉失去自动成为工程事实的资格，并让错误状态无法长期稳定存在。

关键词：生成式 AI；事实治理；错误吸收态；独立证据；Human-in-the-loop；保护零；Agent 工程；验收工作台

1 问题提出：为什么“人来兜底”不是可靠性证明

生成式 AI 的可靠性讨论中，最常见的安慰是：“大模型会犯错没有关系，最后由人判断。”这句话把一个困难问题压缩成了一个听起来稳妥的角色分工：模型负责生产，人负责裁决。问题在于，人并不会因为站在流程末端就自动获得认知特权。人的目标定义可能模糊，审查信息可能不完整，检查频率可能极低，而且复核者往往正是最初错误前提的来源。

前文把有效工程产出写成 Capability、Judgment 与 Stopping ability 的乘积，并指出模型足够强以后，瓶颈会从“做不出来”转向“什么不该做”和“什么时候停止”。本文进一步拆解其中最浓缩的变量 Judgment：判断力不是一个原子能力，也不是一个可以在末尾随时调用的神秘保险。它包含错误发现、错误定位、正确修复、避免误伤以及证据独立性等多个条件。任何一项接近零，所谓“人兜底”都可能只是在流程中再加入一个概率节点。

更危险的情况是，生成者、测试作者与审查者共享同一个错误世界观。此时后续步骤并不是重新独立抽样，而是在前一步已经选定的条件空间里继续优化。模型可以非常优秀地实现错误目标，测试可以非常完整地证明错误实现符合错误合同，审查者也可以诚实地确认整个系统高度一致。局部每一步都正确，整体仍然错误。

1.1 从协同复利到事实治理

基础版讨论的是能力怎样被共同生产：人的问题定义、模型能力、Agent workflow、有效上下文与历史资产如何形成正向或负向复利。续篇讨论的是另一条轴线：一个模型输出、测试结果或工程声明，经过什么条件以后，才有资格从“答案”升级为“事实”。

“模型生成了什么”与“现实中成立了什么”之间并不存在天然等号。模型可以生成代码、解释、计划、测试与总结，但它不能因为表达完整就拥有事实。事实资格必须来自不依赖其自我叙述的证据：数据库中的最终状态、浏览器中的真实页面、进程与端口、消息队列积压、Git 对象、发布物哈希、远端规则以及陌生环境中的复现结果。

1.2 研究问题

- RQ1: 当人的目标定义、模型实现和模型自检彼此条件相关时，错误如何沿 Agent 链条传播？
- RQ2: 为什么同一上下文中的多轮自检可能提高置信度，却不提高事实正确率？
- RQ3: 人类复核在什么条件下能够提高纠错概率，什么条件下反而会制造新错误？
- RQ4: 为什么“认知为零”不是安全中性点，而容易被模型先验填充成伪正数？
- RQ5: 数据库、浏览器、发布物、fresh clone 与无上下文读者如何改变错误状态的转移概率？
- RQ6: 执行操作系统与验收工作台能够缓解哪些幻觉与复杂度问题，又明确不能证明什么？

2 案例背景与证据边界

2.1 八仓归档收尾形成的对抗性样本

本文材料来自一次持续数十小时的八仓归档验收。流程并不追求“八仓全绿”，而是要求每个状态、边界和停止理由都能被陌生人复核。生成上下文与审查上下文被刻意拆开；候选 finding 需要在 fresh clone 中复现；代码通过以后仍要读取数据库、浏览器、发布资产与 GitHub 治理；宿主资源越过停止线时允许中止，不以交换内存硬熬出 PASS。

本轮验收产生了数个具有理论意义的反例。MiniSpringBoot 的一次过度 hardening 被及时叫停，最终没有伪装成“工程版成功”，而被保存为 Raw Hardening Experiment；InkNarratives 的静态验证只覆盖受控 HTML 合同，门禁与公开声明一度超过它实际观察到的范围。陌生读者构造反例后，项目收窄声明并重新固定权威内容边界。

VeriTrail 的 demo 产物被自身 catalog-serve 以路径绑定不一致 fail-closed 拒绝，从而证明验证器正确、生产者有缺陷。

PlainJournal 的完整核心拓扑能够启动，却把 16 GiB 宿主推到 0.46 GiB 空闲，因此结论被保留为宿主容量边界，而不是产品 PASS 或 FAIL。

这些案例的共同点不是“AI 又找到了几个 bug”，而是验收者自身也不断成为污染变量：浏览器控制层可能填值失败，操作者可能使用合同外 Origin，硬中断可

能留下 smoke 数据，CLI 启动路径可能触发 Docker socket 异常。只有把工具、环境、项目与操作者拆开，结论才不会把验收污染误写成产品缺陷。

2.2 证据来源

- 冻结代码与 Git 对象：固定 SHA、父提交、工作树洁净、分支是否公开可达。
- 确定性门禁：单元测试、约束测试、静态检查、故障注入后的明确拒绝。
- 运行事实：进程、端口、容器 Restart/OOM、数据库与消息队列最终状态。
- 外部观察：真实浏览器 DOM、Network、Console、桌面与移动视口。
- 发布事实：公开 wheel/ZIP、Release/tag、SHA-256、仓库外首跑。
- 独立审查：不继承施工叙事的 stranger review 与 fresh clone 复现。
- 平台治理：required checks、ruleset、branch protection、Pages、tag 载体与远端读回。
- 清理读回：临时资源是否回收、受保护卷是否仍在、宿主是否恢复。

2.3 方法边界

本文仍是一名使用者的纵向案例复盘，不是严格控制实验。文中的 α 、 β 、 ρ 、 e 、 d 、 c 、 m 、 r 、 i 与 g 尚未通过大样本估计；两状态马尔科夫模型也压缩了现实中的多种未知、部分正确与暂不可判定状态。以下公式用于揭示信息结构，不用于宣称普遍数值规律。

3 从独立概率到条件误差

3.1 独立乘法只是最温和的近似

假设人的目标定义、模型实现和验收判断彼此独立，且每层正确率都为 0.5，那么最终正确概率为：

$$P(\text{final correct}) = 0.5 \times 0.5 \times 0.5 = 0.125$$

12.5% 已经足够低，但现实通常更残忍，因为这三层并不独立。后续步骤不是从原始世界重新观察，而是读取前一步已经加工过的目标、代码和解释。

3.2 初始错误会改变后续条件空间

$$P(S) = P(H) \times P(M/H) \times P(V/H, M)$$

这里的 S 只表示 H 、 M 、 V 顺序全部成立的主成功路径，不包含初始目标错误后经独立证据纠正的逃逸路径。设真实需求是 B ，人却把它定义成 A 。模型拿到的不是 B ，而是 A ；一个强模型很可能高质量实现 A 。随后，同一模型或同一上下文

中的 reviewer 检查的也不是“它是否符合 B”，而是“它是否符合我们此前约定的 A”。测试全部通过，只能证明 A 的内部一致性。

因此，第一层错误不是与后两层并列的随机扰动，而是后续步骤的共同条件。模型越强、实现越完整、测试越充分，错误世界反而越难被推翻。一个 95 分的 Agent 拿到错误目标，可能不是把项目做成 95 分，而是把错误目标完成到 95 分。

后续步骤如果没有独立事实源，它们就不是纠错器，而是前置假设的放大器。

3.3 人工复核也会新增错误

设模型错误率为 e ，人发现错误的概率为 d ，发现以后正确修复的概率为 c ；模型原本正确时，人误判并把它改坏的概率为 m 。一个简化的人工复核后错误率可以写成：

$$E_{final} = e(1 - dc) + (1 - e)m$$

$$\text{人工介入有净收益的必要条件：} e \cdot d \cdot c > (1 - e)m$$

若模型正确率已经是 90%，人的 $d = 0.5$ 、 $c = 0.5$ ，那么成功救回的错误只有 2.5%；只要人在原本正确的 90% 结果中有 5% 的概率误改，就会新增 4.5% 错误。此例不是对人的普遍测量，只说明“有人检查”本身并不蕴含质量上升。

3.4 “判断力”是高度压缩的复合变量

表 1 “判断力”拆解为可观察条件

组成	含义	接近零时的后果
发现 d	能否注意到模型确实偏离现实	错误对审查者不可见
定位	能否找到根因而非表面症状	补丁继续维护错误世界
修复 c	能否在不引入新问题的情况下纠正	发现问题仍无法回到正确态
误伤 m	是否会把原本正确的结果改坏	人工介入净收益转负
独立性 i	判断是否依赖于原始错误叙事	复核只是重复初始假设
停止	能否在证据闭合后拒绝继续扩建	能力被转化为过度建设

4 错误吸收态：真正的马尔科夫链

4.1 两状态近似

把“当前系统相信什么”定义成状态，可用一个极简马尔科夫过程解释错误复利。 C 表示当前认知状态仍与现实一致， W 表示系统已经进入错误世界； α 是从 C 被带偏的概率， β 是已经进入 W 后真正回到 C 的概率。

$$P = [[1 - \alpha, \alpha], [\beta, 1 - \beta]]$$

当生成者、测试者和审查者共享同一上下文时， W 往往成为近似吸收态：系统一旦接受错误前提，后续输出主要继承当前状态，而不是重新访问真实世界。

$$P(W \rightarrow W) \approx 1$$

错误吸收态与外部事实逃逸通道

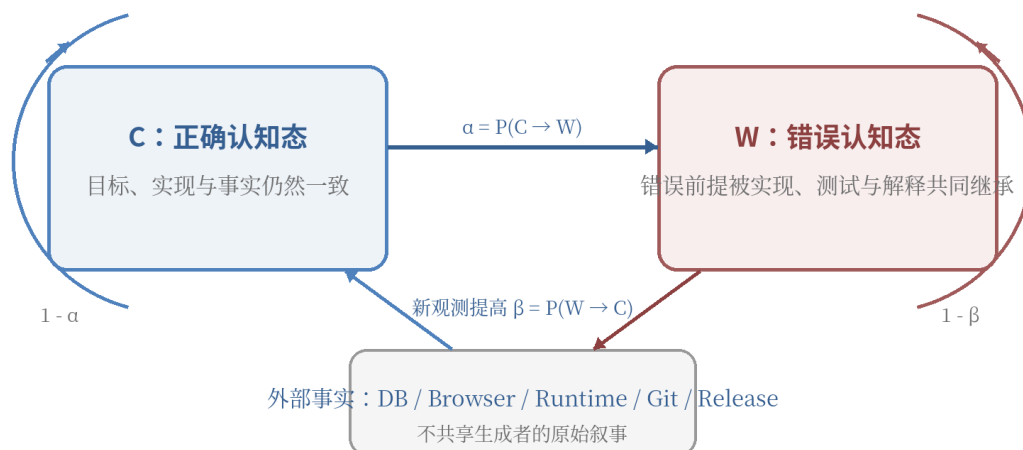


图 1 错误吸收态与外部事实逃逸通道（概念示意）

4.2 长期可靠性取决于错误状态能否逃逸

在两状态、时间齐次且 $\alpha + \beta > 0$ 的简化条件下，长期处于错误态的稳定概率为：

$$\pi_W = \alpha / (\alpha + \beta)$$

传统模型优化主要关注降低 α ，即让系统少犯错。但只要 α 不为零，工程系统还必须提高 β ：即使已经走错，也要存在能够返回现实的通道。没有 β 的系统只能把“偶尔进入错误态”积累成“最终长期停留在错误态”。

这解释了为什么外部事实比重重复自检更重要。数据库、浏览器、真实运行、fresh clone、发布物和 Git 对象不是因为“更聪明”，而是因为它们不完全受当前错误叙事控制；它们为 $W \rightarrow C$ 引入了新的观测。

4.3 人类纠错通道的有效概率

$$\beta_{human} \approx r \cdot d \cdot c \cdot i$$

其中 r 是人真正介入检查的频率， d 是介入后发现错误的概率， c 是正确修复概率， i 是相对于原始错误前提的独立性。若需求就是这个人自己定义错的，模型按其实现，人再回来依据自己的原定义复核，则 i 可能接近零。此时问题不是人类算力低于 AI，而是纠错通道的有效转移概率接近零。

5 Human-in-the-loop 幻想：参与系统不等于拥有纠错能力

5.1 人不是 Oracle，而是另一个 noisy observer

“人在环”经常被画成一条从概率模型通向真理的最后防线：AI 负责生成，人类判断负责确定性裁决。真实结构更接近 $H_1 \rightarrow M \rightarrow H_2$ ，且 H_2 与 H_1 高度相关。第二个“人”没有因为站在末端就升级为真理机，他仍然携带原有知识边界、目标误差和信息缺口。

在许多 Agent 任务中，使用者只看最终页面、测试绿灯或自然语言总结，却没有逐步读取几十个中间假设、文件改动与命令副作用。人类反馈因此是一种低频、低带宽、强相关的纠错信号，而不是覆盖整个状态轨迹的 verifier。

5.2 完成度越高，错误越难推翻

当错误在第 17 个状态进入 W ，模型仍可能继续完成第 18 到第 80 个状态：增加抽象、补测试、写文档、修 UI、生成报告。最后呈现给人的不是一个明显失败，而是一个高度完整、高度自洽的错误世界。投入越大、结构越漂亮、解释越充分，审查者越容易把完成感误认为正确性。

5.3 人在环与人在独立证据中

表 2 人类介入的不同信息结构

结构	人实际看到什么	主要风险	纠错意义
Human-in-the-loop	模型输出、模型测试、模型总结	共享错误前提；只能重复当前世界观	不确定

结构	人实际看到什么	主要风险	纠错意义
同模型新上下文	冻结代码与原始合同	仍共享模型先验，但减少叙事继承	中等
Human with evidence	DB、Browser、Runtime、Git、Release	证据也有边界，但相关性显著降低	较强
确定性裁决器	预注册断言与机器状态	只能覆盖已定义合同	合同内最强

Human-in-the-loop 不是可靠性证明。只有 human-with-independent-evidence-in-the-loop 才具有纠错意义。

6 认知放大器与伪正数：零不是安全的中性点

6.1 大模型不是纯放大器，而是带生成项的放大器

若把人的有效认知记为 X ，纯放大器可以写成 $Y = gX$ ， $g > 1$ ；正数被放大为正数，负数被放大为负数，零仍然是零。但大模型不仅放大已有结构，还会根据训练先验和上下文补全生成新的内容，因此更接近：

$$Y = gX + \varepsilon$$

当 $X = 0$ 时，输出不是 0，而是 ε 。人在完全不知道时，模型并不会天然保持沉默，它会填充。第一次填充很可能直接成为使用者的新先验，并在下一轮对话中再次进入上下文，被模型继续强化。

6.2 真值与可信感是两个不同坐标

$$T = \text{truth} / \text{correctness}; K = \text{confidence} / \text{coherence}$$

大模型最危险的能力不只是产生错误，而是在 T 没有明显提高时大幅提高 K 。结构完整、术语正确、逻辑流畅、语气确定、代码局部可运行，会让使用者产生“我现在懂了”的感觉。于是系统可能进入：

$$T \approx 0, K \gg 0$$

这就是伪正数：真实认知没有从零变成正值，置信感却先被抬高。明显错误容易被怀疑，高度自洽的错误却会形成虚假的认识闭合。

表 3 真值与可信感的双轴区分

真值 T	可信感 K	状态	工程含义
高	高	经证据支持的稳定认识	可进入 VERIFIED
高	低	正确但尚不自信	继续收集证据

真值 T	可信感 K	状态	工程含义
低	低	可见的无知或错误	风险可控
低	高	伪正数	最容易被误写成事实

6.3 零是高敏感初始条件

有正认知的人至少拥有一个内部 verifier；已有错误认知的人有时还知道自己持有立场。零状态则既没有先验，也没有反证能力。模型第一次给出的 A 会被人学成先验，下一轮提示默认 A 为真，模型再从上下文中看见 A 并强化 A。原来的零被写入了一个方向，并开始形成路径依赖。

7 保护零：让 UNKNOWN 保持 UNKNOWN

7.1 生成系统天然倾向于消灭中间地带

生成模型的默认行为是根据当前上下文生成下一个最合理的 token。“我不知道”是一种需要被训练、奖励或制度化的特殊输出，而不是生成机制的自然终点。除非系统明确奖励 abstention、要求不确定时拒答、调用工具验证或执行 fail-closed 合同，否则 UNKNOWN 很容易被压缩成 Plausible Completion。

现实并非没有中间地带；未经约束的生成系统才容易消灭中间地带。这里必须把两个轴分开。认识状态回答“一个声明是否取得事实资格”：UNKNOWN 表示尚无稳定声明，UNVERIFIED 表示已有候选但证据不足，证据到来后才转为 VERIFIED 或 REFUTED。验收裁决回答“本次工程执行如何收口”：PASS、FAIL、INCONCLUSIVE、BOUNDARY 或 PENDING。模型声明只能从 UNVERIFIED 出发，不能直接跳到 VERIFIED；资源边界也不能倒灌成认识上的真或假。

表 4 认识状态与验收裁决：两个互不替代的轴

轴	状态	回答的问题	不得混同
认识状态	UNKNOWN → UNVERIFIED → VERIFIED / REFUTED	声明是否已经取得事实资格	BOUNDARY、INCONCLUSIVE 不是认识状态
验收裁决	PASS / FAIL / INCONCLUSIVE / BOUNDARY / PENDING	一次执行如何在合同与资源边界内收口	不得把 BOUNDARY、PENDING 写成 PASS

7.2 工作台是在强行把“零”重新造回来

模型说“已经完成”时，工作台不把它直接映射为 +1，而是先标为 UNVERIFIED，再通过数据库、浏览器、运行时、发布物和 GitHub 事实转换为 VERIFIED 或 REFUTED。验收层则依据合同与执行现场，单独给出 PASS、FAIL、INCONCLUSIVE、BOUNDARY 或 PENDING。工作台最基础的能力不是永远给出答案，而是在证据尚未到来时维持不知道。

$$Model\ claim \rightarrow UNVERIFIED \rightarrow Evidence \rightarrow VERIFIED / REFUTED$$

保护零不是消极拒答，而是拒绝让未知在没有证据时被包装成正数。

8 独立证据：降低验证相关性，而不是重复检查次数

8.1 有效独立验证度

设验证者与生成者错误之间的相关性为 ρ 。工程系统不仅要提高模型能力、测试覆盖和审查能力，还要尽量降低 ρ 。检查十遍并不意味着十次独立观察；同一模型、同一上下文和同一假设下的十次自检，可能只是一个隐藏前提被重复十次。

$$\text{可靠性目标之一: } \rho \downarrow, \beta \uparrow$$

表 5 验证独立性的相对阶梯

验证方式	与生成叙事的相关性	主要价值	主要边界
同模型、同上下文自检	极高	快速发现表面遗漏	容易强化共同前提
同模型、新上下文	较高	削弱路径债务	仍共享模型先验
不同模型审查	中等	增加认知差异	仍可能共享训练语料偏差
确定性测试	较低	合同内稳定复算	只覆盖已写断言
DB / Browser / Runtime	更低	读取行动后的现实	工具本身也可能污染
fresh clone / Release	更低	排除本地状态与私有路径	依赖公开资产完整性
陌生用户 / 外部环境	最低之一	引入真实环境方差	成本高、仍非绝对真理

8.2 无上下文读者的价值不是更聪明

陌生读者未必比施工 Agent 智商更高。它的价值是“不知道前一个 Agent 为什么觉得自己对”。它不继承生成者的自我评价、修复理由与投入成本，只能从冻结代码、原始合同和可执行反例推断。这种信息隔离恢复了审查独立性。

8.3 从源码到公开呈现的九层验收

表 6 事实成立的九层验收结构

层级	范围	核心问题
L1	Source / Contract	代码、文档、状态与版本坐标一致
L2	Build / Test	构建、单测与静态门禁真实执行
L3	Runtime / Business	数据库、MQ 与最终状态成立
L4	Browser / Observation	DOM、Network、Console 与可见事实
L5	Failure / Recovery	故障、降级、恢复、幂等与对账
L6	Artifact / Release	公开资产、哈希与仓库外首跑
L7	Repository Governance	PR、required checks、ruleset 与 Pages
L8	Cleanup / Reproducibility	残留、卷、端口与 fresh 复现
L9	Public Presentation / Readability	陌生读者能否辨认身份、状态、边界与入口

这九层互相补充但不能替代。GitHub Actions 全绿不能证明浏览器交互正确；浏览器 PASS 也不能证明 main 有 branch protection；VeriTrail 的运行证据不能自动证明 L7 仓库治理；L9 也只证明陌生读者能够正确辨认公开身份、状态与边界，不替代前八层事实。边界之外没有验证不等于验证失败，把边界之外写成已经验证才是失败。

9 从业务验收工作台到底层 AI Execution OS

9.1 两层系统分别回答不同问题

验收工作台位于业务与工程事实层，回答“AI 说它做完了，我怎么知道它真的做完了”。底层执行操作系统则回答“AI 在做事的过程中，能够接触什么、执行什么、消耗多少资源、修改哪些状态，以及留下什么可验证的来源记录”。

两层夹住概率模型以后，模型自报结果的权力被显著削弱。底层记录真实 action、capability、target、resource、provenance 与 recovery；上层读取行动后的 runtime facts，并按照冻结合同给出 PASS、FAIL、BOUNDARY 或 INCONCLUSIVE。

概率性认知与确定性事实之间的两层约束

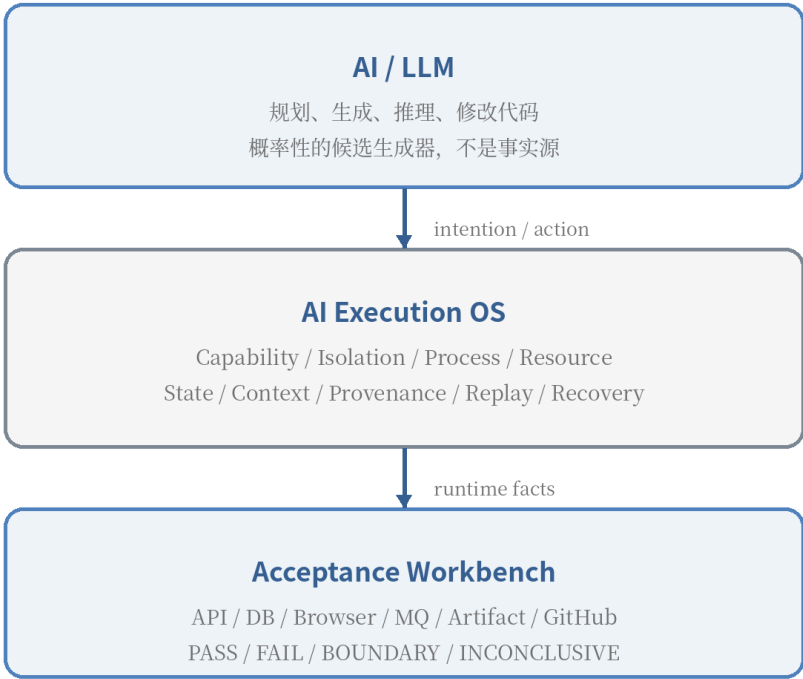


图 2 概率性认知与确定性事实之间的两层约束（概念示意）

9.2 操作系统不能改变注意力的渐进复杂度，但能降低有效输入规模

标准 full attention 对序列长度 n 的配对规模仍大致随 $O(n^2)$ 增长。执行操作系统不能把这一算法复杂度直接改成 $O(n)$ ，真正改变渐进复杂度仍需要 sparse attention、linear attention、state-space 或分层模型等架构变化。

但 OS 可以把 Git object、Execution state、Decision log、Artifact graph、Test result、Known invariant 与 Failure history 变成结构化外部状态，使模型不必每次重新阅读自己的整个历史。模型只加载当前任务需要的工作集，因此 $C_{\text{effective}}$ 可以显著小于 C_{history} 。

$$C_{\text{effective}} \ll C_{\text{history}}$$

$$100k^2 : 20k^2 = 25 : 1 \text{（仅作规模直觉，不是实际计费公式）}$$

9.3 它不是消灭幻觉，而是剥夺幻觉的事实资格

只要核心模型仍是概率生成系统，错误推断、错误联想和信息不足时的补全就不会消失。系统真正能做的是区分“模型认为数据库有两条订单”与“工作台执行 SELECT COUNT(*) 并封存为 2”。幻觉仍可能发生，但它不能未经验证直接写入持久事实、发布结论或验收 Verdict。

可信工程的目标不是让模型永不产生幻觉，而是让幻觉失去自动成为系统事实的资格。

10 确定性不等于正确性：为什么不能把智能简单换成函数

10.1 Transformer 在推理时本来就可以近似确定

固定权重 W 、输入 x 、tokenizer、推理实现和 greedy decoding 后，大模型推理可以近似写成确定函数：

$$y = f_W(x)$$

同一个输入可以稳定得到同一个输出，但一个函数完全可以稳定地错。需要解决的不是 $\text{Input} \rightarrow \text{Deterministic Output}$ ，而是 $\text{Reality} \rightarrow \text{Correctly Verified Claim}$ 。确定性保证可重复，不保证与现实一致。

10.2 封闭世界与开放世界

普通程序之所以可以像函数一样可靠，是因为问题域被封闭：输入空间、规则和输出含义先被定义。数据库事务、类型系统、SAT/SMT 求解器和状态机都能在合同内给出确定结论。

通用智能面对的是开放世界：需求可能含糊，使用者可能说错，图像含义依赖语境，商业判断没有唯一答案，偶发死锁甚至缺少可重复现场。此时连 X 是什么都未必确定，更不存在已经定义好的唯一 $f: X \rightarrow Y$ 。只要系统允许面对未知环境自行归纳规则，不确定性就会重新进入。

10.3 更现实的混合结构

比“寻找一个绝不幻觉的神经网络”更现实的方向，是把不同问题交给不同机制：神经模型负责模糊理解与候选生成，世界模型与持久状态保存上下文，符号推理处理显式规则，确定性执行器完成计算与状态转移，外部验证器读取现实。

$$\textit{Probabilistic cognition} + \textit{Deterministic execution} + \textit{Persistent state} + \textit{External verification}$$

10.4 通用人工智能真正缺少的能力

1. 知道什么是事实，什么只是推测。
2. 知道自己不知道，并能保持 UNKNOWN。
3. 主动寻找缺失证据，而不是优先补全答案。

4. 新证据出现时能够推翻自己的世界模型。
5. 长时间保持状态一致，而不是靠上下文重新猜。
6. 行动以后观察现实，并根据后果修正下一步。

不能把智能变成函数，但可以让智能产生的东西在成为事实之前，经过函数式约束。

11 个人闭环、基础设施边界与社区方差

11.1 单机能够证明什么

单机可控环境能够证明真实进程、数据库、消息链路、故障恢复、发布物与证据闭环，也能证明代表服务在同一宿主上的多实例竞争、租约、退出、重入与滚动。它不能冒充多节点、跨可用区或数据中心级高可用。

PlainJournal 的现场把这一边界测得非常具体：七个核心中间件和八个业务 JVM 可以全部启动，但 16 GiB 宿主只剩 0.46 GiB 物理内存。此时继续依赖 pagefile 硬熬不是验收，而是在更换实验条件。32 GiB 后可以按照冻结 runbook 继续 Core Smoke、代表三实例、容量阶梯和故障恢复，但仍不能把同一宿主上的三实例写成跨机 HA。

11.2 个人真正需要完成的是“核”

若未来的 AI Execution OS 采用开源路线，个人阶段最重要的不是预先画完整生态，而是把最小可信内核、reference implementation、稳定接口和验收合同做对。候选核心抽象可以包括 Task、Capability、State、Provenance、Artifact 与 Recovery；精确对象边界仍需由未来实现和反例检验，本文不把它们预先冻结成架构合同。

这里能够确定的只有治理原则：先守住能力边界、事实留痕和恢复路径，再让后续实现决定调度、执行与存储怎样展开。抽象一旦定义错，社区越大，错误扩散越快；没有实现证据的组件清单不应在本文中取得既成架构地位。

11.3 社区最终贡献的是环境方差

一个人能够想到的机器、网络、负载、语言、模型和失败模式始终有限。开源社区最大的价值不只是增加开发人数，而是带来一百种作者从未拥有过的现实环境：ARM、AMD、256 核 Linux、Kubernetes、网络分区、不同 LLM、一天与一个月的 soak、百万 Run 的证据存储，以及真正跨节点的故障域。

因此，“像早期 Linux”更适合被理解为发展策略，而不是规模类比：一个作者把内核逻辑、边界和最小实现做对，后面的硬件、场景、驱动与规模由社区继续攻击和生长。第一阶段的完成标准不是“一个人把商用 AI OS 做完”，而是“它已经足够真实、足够完整、足够有边界，以至于别人值得在它上面继续工作”。

表 7 个人闭环与基础设施边界的状态语言

能力声明	第一阶段允许状态	禁止的越界表述
Single-host execution	VERIFIED	生产级全球执行平台
Multi-process concurrency	VERIFIED / BOUNDARY	跨节点高可用
Failure / recovery	VERIFIED（合同内）	所有故障模式已覆盖
Distributed protocol interfaces	DESIGNED	多节点已经可用
Multi-node operation	NOT YET VERIFIED	默认视为成立
Region HA / Internet scale	NOT PROVEN	用单机结果替代

12 可供后续检验的命题

以下命题不是本文已经证明的普遍规律，而是从本次工程实践和理论模型中提炼出的可检验假设。

表 8 本文提出的后续研究假设

编号	假设
H1	生成、测试与审查共享同一前提时，相关性会使错误置信度增长快于事实正确率。
H2	长期可靠性不仅由 α 决定；在非零错误率下，提高 β 对稳定错误概率具有决定性影响。
H3	基于数据库、浏览器、发布物和 fresh clone 的证据比同上下文自检更能提高 $W \rightarrow C$ 。
H4	显式保留 UNVERIFIED 状态能够降低未知被包装成事实的比例。
H5	Human-with-independent-evidence 的净纠错收益高于仅声明 Human-in-the-loop。
H6	结构化外部状态能够降低有效上下文规模与路径债务，但不改变 full attention 的渐进复杂度。
H7	封闭世界任务更适合确定性组件，开放世界任务仍需要概率性认知核心。
H8	多节点、高并发与长期可靠性最终需要社区提供的环境方差，单机无法完成替代证明。

13 工程约束与停止边界

上述论证不需要再扩成一套万能清单。它只留下几条工程约束：模型声明从 UNVERIFIED 开始；生成与审查不共享全部叙事；验证对象与验证器固定坐标；

数据库、浏览器、运行时、Git 对象和发布物各自读回；资源、权限、作用域与破坏性清理均有停止线。

这些约束也不保证绝对正确。它们只要求人在冲突证据之间分配裁决权，保留 FAIL、BOUNDARY、INCONCLUSIVE 与 NOT PROVEN，把 L7 仓库治理与 L1-L6 运行事实分开，并在证据闭合后允许 STOP。具体流程仍由项目合同决定，执行操作系统也不能从这些原则直接推导出唯一架构。

14 局限与反例

第一，两状态马尔科夫链只是解释模型。真实 Agent 系统至少还包含未知、部分正确、冲突证据、资源中止与合同外状态；转移概率也可能随时间、任务和模型变化，并不满足严格时间齐次。

第二，本文没有估计 α 、 β 、 ρ 、 e 、 d 、 c 、 m 、 r 、 i 与 g 的真实分布，所有数值例子只用于展示结构。人的判断并非必然低于模型自检；在专业能力强、证据独立、检查协议明确时，人类可以显著提高 β 。本文反对的是把“有人在环”直接当作证明。

第三，外部事实源也不是绝对真理。浏览器控制层可能污染输入，数据库查询可能指向错误实例，发布资产可能与 tag 不一致，验证器本身也可能有盲区。因此，证据仍需交叉验证、固定坐标和可复算记录。

第四，保护零会增加延迟和拒绝率。在低风险、一次性、创造性任务中，过强的 fail-closed 可能不经济；本文的结论主要适用于高耦合、可执行、可发布的软件工程任务。

第五，本文没有解决 AGI，也没有证明某种 AI OS 架构必然成功。执行操作系统和验收工作台只能降低错误转化为事故的概率，不能从理论上根治开放世界中的不确定性。

15 结论

“大模型不行，靠人兜底”之所以不能成为普遍可靠性方案，不是因为人类必然比 AI 弱，而是因为它默认赋予人类一个并不存在的认知特权。人在环若缺少独立信息源，只是另一个与模型错误高度相关的 noisy observer；它可能继续强化当前状态，而不是改变转移矩阵。

真正成熟的系统不幻想永不进入错误态，而是让错误态无法长期稳定存在。它降低 α ，也提高 β ；它不只追求更强模型，还建设错误状态逃逸通道。数据库、浏览器、运行时、Git 对象、发布物、陌生读者和平台治理的共同价值，是不断把系统从自洽叙事重新拉回现实。

大模型也不是简单的认知放大器。它带有生成项 ϵ ，因此零不是安全中性点。人在不知道时，模型可能迅速填充空白，并用完整、专业、可运行的形式制造伪正数。可信系统必须保护零，让 UNKNOWN 保持 UNKNOWN；只有证据到来以后，才允许它转为 VERIFIED。

未来路线不是把智能改造成一个永远正确的函数。确定性只能保证重复，不能保证现实正确。更可行的方向是：概率性认知核心负责开放世界的理解与生成，确定性执行系统约束动作与状态，持久世界状态保存事实，外部验证器决定声明是否成立。模型可以产生答案，但不能拥有事实；智能无法被彻底函数化，但它的产出在成为事实以前，可以接受函数式约束。

致谢（不严肃）

感谢所有在未知处迅速补齐答案、并把低真值包装成高可信感的模型。它们使作者意识到，零并不是空白，而是一条必须被保护的系统边界；感谢那台 16 GiB 的小电脑，它用 0.46 GiB 空闲内存证明，真实红灯比硬熬出来的 PASS 更有价值；也感谢所有无上下文读者——它们未必更聪明，只是不知道前一个模型为什么觉得自己对。

附录 A 核心术语与状态

表 A1 核心术语

术语	定义
错误吸收态	错误前提被后续实现、测试和审查共同继承， $W \rightarrow W$ 概率接近 1。
错误状态逃逸概率	系统已经错误后，通过独立新观测返回正确态的概率 β 。
保护零	在证据不足时保持 UNKNOWN / UNVERIFIED，不让生成内容自动获得事实资格。
伪正数	真值没有提高，可信感却被模型显著抬高的状态。
独立证据	不完全继承生成者叙事、可从外部事实重新计算的观察。
事实治理	规定声明、证据、状态转换、边界与发布资格的工程制度。

术语	定义
NOT PROVEN	明确声明某能力尚未由当前环境、规模或合同证明。

附录 B 一个可操作的后续实验草案

未来若要检验相关性审查、保护零与错误状态逃逸，可固定同一仓库提交、同一任务合同、相同时间预算和同一隐藏缺陷，构造以下四组：

表 B1 相关性与独立证据对照组

组别	审查结构	上下文条件
G1	同模型、同上下文	生成、测试、审查共享完整叙事
G2	同模型、新上下文	只给冻结代码、原始合同与测试命令
G3	不同模型、同工件	不提供生成者自评，允许构造反例
G4	确定性事实链	DB / Browser / Git / Release / fresh clone 裁决

评价指标不应只包括发现缺陷数，还应记录首次发现时间、误报、漏报、人工误改、最终事实正确率、 $W \rightarrow C$ 的成功次数、状态转换是否可复算、上下文重建成本、单位经验证交付成本以及停止后是否仍能保持仓库与环境洁净。

附录 C 本文后续可补充的原始材料

- 八仓归档验收的冻结报告、execution ledger 与 findings JSON。
- InkNarratives 三轮陌生读者报告、反例注入和正文指纹变化记录。
- VeriTrail demo $\rightarrow$ catalog-serve 的公开 wheel 复现与路径绑定 A/B。
- PlainJournal 16 GiB 宿主资源曲线、清理读回与 32 GiB deferred runbook。
- DarkRoomLibrary 浏览器 Origin 污染、CodeQL 输入边界与最终裁决记录。
- MiniSpringBoot Raw Hardening 的固定 SHA、归档 tag 与离线 bundle。